Manual de Programación del Lenguaje K++

De cero a tus primeros programas que compilan a KolibriOS

Versión del lenguaje documentada: **K++ 0.9** (compilador 0.4)

Autor del lenguaje y compilador: **Esequiel Pena**

Fecha del manual: octubre de 2026

⚠ K++ es un lenguaje experimental en desarrollo activo (Stage 0).
Este manual describe únicamente la sintaxis verificada por la suite de pruebas L01–L16.

Índice

1. Introducción: ¿qué es K++?
2. Tu primer programa
3. Estructura de un programa K++
4. Tipos de datos
5. Variables y asignación
6. Operadores
7. Salida por pantalla: `print` y `print_int`
8. Control de flujo: `if` / `else` y `while`
9. Funciones
10. Estructuras: `struct`
11. Enumeraciones: `enum`
12. Comentarios
13. Errores comunes y mensajes del compilador
14. Compilar y ejecutar tu programa
15. Limitaciones conocidas (lo que K++ aún no hace)
16. Referencia de ejercicios verificados L01–L16

1. Introducción: ¿qué es K++?

K++ (archivos `.kpp`) es un lenguaje de programación experimental creado desde cero, junto con su propio compilador: lexer, parser, analizador semántico, representación intermedia (IR), backend x86 y generador de ejecutables propios. No es una envoltura sobre otro compilador.

Su objetivo es ambicioso y poco común:

- Compilar programas escritos en un subconjunto de C++ a **código máquina x86 de 32 bits**.
- Generar ejecutables **nativos de KolibriOS** (archivos `.bin`), no ELF de Linux.
- Crecer por etapas hasta lograr el **self-hosting**: que el compilador de K++ esté escrito en K++ y pueda compilarse a sí mismo.

Hoy K++ está en **Stage 0**: el subconjunto mínimo del lenguaje ya compila en Linux (bootstrap) y genera binarios que corren en KolibriOS. Este manual documenta exactamente ese subconjunto, verificado ejercicio por ejercicio con el compilador real.

2. Tu primer programa

El clásico punto de partida. Guárdalo como `hola.kpp`:

```
int main()
{
    print("Hola K++");
    return 0;
}
```

Y compílalo:

```
./bin/kpp hola.kpp
```

Si todo va bien, el compilador muestra su cabecera, el árbol sintáctico (AST) del programa y genera el ejecutable. `print("...")` escribe texto en pantalla y `return 0` indica que el programa terminó correctamente.

3. Estructura de un programa K++

Un programa K++ se compone de **declaraciones globales**, que solo pueden ser de tres tipos:

- declaraciones `struct`,
- declaraciones `enum`,
- funciones que devuelven `int`.

Todo programa necesita una función `main`: es el punto de entrada por donde empieza la ejecución.

```
struct Persona
{
    int edad;
    string nombre;
};

enum Color
{
    ROJO,
    VERDE,
    AZUL
};

int main()
{
    print("Estructura completa");
    return 0;
}
```

> **Nota:** las llaves `{ }` delimitan los bloques de código y cada instrucción termina con punto y coma `;`, como en C++.

4. Tipos de datos

K++ en Stage 0 trabaja con dos tipos de datos:

Tipo	Descripción	Ejemplo
`int`	Número entero de 32 bits	`int edad = 25;`
`string`	Cadena de texto (literal entre comillas dobles)	`string nombre = "Ana";`

> **Importante:** no des por hecho que existen `void`, `bool`, `float`, `double`, `char` ni variantes como `long` o `unsigned int`. Están en la hoja de ruta, pero el lenguaje operativo de hoy trabaja principalmente con `int` y `string`.

5. Variables y asignación

Las variables se declaran con su tipo y pueden inicializarse en la misma línea:

```
int main()
{
    int a = 10;
    int b = 20;
    int c = a + b;

    print("L02 VARIABLES");
    print_int(a);
    print_int(b);
    print_int(c);

    return 0;
}
```

También puedes asignar un nuevo valor a una variable ya declarada (sin repetir el tipo):

```
int contador = 3;
contador = contador - 1;
```

> **Regla:** no puedes usar una variable antes de declararla, ni declarar dos veces el mismo nombre en el mismo ámbito. El analizador semántico lo detecta y el compilador lo rechaza.

6. Operadores

6.1. Aritméticos

Operador	Significado	Estado
+	Suma	✅ Funciona
-	Resta	✅ Funciona

`*`	Multiplicación	✅ Funciona (el backend emite `imul`)
`/ %`	División, módulo	❌ No implementados: el compilador los rechaza con error, no genera código incorrecto

```
int a = 20;
int b = 6;
print_int(a + b);   /* 26 */
print_int(a - b);   /* 14 */
print_int(a * b);   /* 120 */
```

6.2. Relacionales (comparación)

Operador	Significado
`==`	Igual a
`!=`	Distinto de
`<`	Menor que
`>`	Mayor que
`<=`	Menor o igual que
`>=`	Mayor o igual que

6.3. Lógicos

Operador	Significado
`&&`	Y lógico
`||`	O lógico
`!`	Negación

```
int a = 10;
int b = 5;

if (a == 10 && b != 0)
{
    a = a + 1;
}

if (a > b || a <= 0)
{
    a = a + 1;
}

if (!(a == b))
{
    a = a + 1;
}
```

7. Salida por pantalla: `print` y `print_int`

K++ ofrece dos funciones integradas de salida (implementadas en su runtime para KolibriOS):

Función	Uso	Ejemplo
`print`	Escribe un literal de texto	`print("Hola K++");`
`print_int`	Escribe un valor entero (variable o expresión)	`print_int(edad);`

```
int main()
{
    int edad = 25;
    string nombre = "Ana";

    print("L01 TIPOS");
    print_int(edad);
    print(nombre);

    return 0;
}
```

Detalle: `print` también acepta variables `string`, y `print_int` acepta expresiones como `print_int(a + b)`.

8. Control de flujo: `if` / `else` y `while`

8.1. Condicionales

```
int clasificar(int x)
{
    if (x > 20)
    {
        return 3;
    }
    else if (x > 10)
    {
        return 2;
    }
    else
    {
        return 1;
    }
}

int main()
{
    print("CASO 25");
    print_int(clasificar(25));   /* 3 */

    print("CASO 15");
    print_int(clasificar(15));   /* 2 */

    print("CASO 5");
    print_int(clasificar(5));    /* 1 */

    return 0;
}
```

La condición va entre paréntesis y el bloque entre llaves. Puedes encadenar `else if` tantas veces como necesites y cerrar con un `else` final.

8.2. Bucles `while`

```
int main()
{
    int contador = 3;

    while (contador > 0)
    {
        print_int(contador);
        contador = contador - 1;
    }

    return 0;
}
```

El bloque se repite mientras la condición sea verdadera. Asegúrate de que algo dentro del bucle modifique la condición, o el programa no terminará nunca.

9. Funciones

Las funciones agrupan código reutilizable. Se definen con tipo de retorno `int`, nombre, parámetros entre paréntesis y cuerpo entre llaves:

```
int sumar(int a, int b)
{
    return a + b;
}

int obtener_valor()
{
    return 42;
}

int main()
{
    int resultado = sumar(10, 20);
    print_int(resultado);        /* 30 */

    int otro = obtener_valor();
    print_int(otro);             /* 42 */

    return 0;
}
```

- **Parámetros:** se declaran con tipo y nombre (`int a, int b`). El compilador valida que la llamada pase la cantidad y los tipos correctos.
- **Argumentos:** pueden ser constantes, variables o expresiones.
- **Retorno:** `return` devuelve un valor entero y termina la función.
- **Llamadas:** el resultado puede guardarse en una variable o usarse directamente, por ejemplo `print_int(clasificar(25));` .

9.1. Expresiones como argumentos

Los argumentos no tienen que ser solo constantes o variables: también pueden ser expresiones, que se evalúan antes de la llamada:

```
int suma(int a, int b)
{
    return a + b;
}

int main()
{
    int x = 10;
    int y = 20;

    print_int(suma(x + 5, y));      /* 35 */
    print_int(suma(x + 5, y * 2)); /* 55 */

    return 0;
}
```

9.2. Llamar funciones definidas después

K++ permite llamar a una función que está definida *más abajo* en el archivo. No necesitas declarar prototipos: el compilador resuelve las referencias hacia adelante:

```
int main()
{
    int x = 0;
    x = suma();
    print_int(x);   /* 20 */
    return 0;
}

int suma()
{
    return 20;
}
```

9.3. Retornar directamente una llamada

Puedes devolver el resultado de una llamada sin pasar por una variable intermedia:

```
int suma()
{
    return 10;
}

int main()
{
    return suma();   /* main retorna 10 */
}
```

PRUEBA DE FUEGO — ayuda de testeo de Stage 0: el compilador imprime automáticamente en pantalla el valor de todo `return` cuya expresión **no** sea una constante literal. Así, `return x;`, `return a + b;` o `return suma();` muestran su valor al ejecutar el programa, mientras que `return 0;` o `return 42;` no imprimen nada. Es una ayuda intencional para validar visualmente los programas en KolibriOS, no un error del compilador.

Dato interno: la convención de llamada propia de K++ pasa el primer argumento en el registro EAX, el segundo en EBX, y devuelve el resultado en EAX. Es un detalle del diseño actual del backend x86, no una ABI estándar.

Límite conocido: las llamadas anidadas como `suma(suma(1, 2), 4)` aún fallan en el backend x86 (prueba L16). De momento, guarda los resultados intermedios en variables.

10. Estructuras: `struct`

Un `struct` agrupa campos con nombre bajo un nuevo tipo. La declaración de estructuras está verificada:

```
struct Persona
{
    int edad;
    string nombre;
};

struct Producto
{
    int codigo;
    string nombre;
};

int main()
{
    print("L07 STRUCT");
    return 0;
}
```

- La declaración termina con punto y coma después de la llave de cierre (`};`).
- El compilador rechaza campos duplicados dentro del mismo `struct` y estructuras con nombres repetidos.

Límite conocido (L13): el acceso a campos con el operador punto (`p.edad`) aún **no** funciona: el lexer no reconoce el carácter `.` y el compilador lo rechaza. La declaración y validación de `struct` sí están verificadas; el acceso a miembros está en desarrollo.

11. Enumeraciones: `enum`

Un `enum` define un conjunto de constantes con nombre:

```
enum Color
{
    ROJO,
    VERDE,
    AZUL
};
```

Puedes asignar valores explícitos; los siguientes continúan la secuencia:

```
enum Estado
{
    Inactivo = 0,
    Activo = 10,
    Pausado          /* vale 11 */
};
```

Y acceder a un valor con su nombre calificado:

```
int main()
{
    return Color::Verde;
}
```

- La declaración termina con `};`.
- El compilador rechaza nombres de enumeradores repetidos y valores inválidos.

12. Comentarios

K++ admite las dos formas clásicas de comentar, igual que C++:

```
int main()
{
    // Esto es un comentario de una línea

    /*
       Esto es un comentario
       de varias líneas
    */

    print("Los comentarios no afectan al programa");
    return 0;
}
```

13. Errores comunes y mensajes del compilador

El compilador de K++ detecta tres familias de errores. Estos son los más frecuentes:

Error	Ejemplo que falla	Mensaje típico
Olvido del punto y coma	`int numero = 10`	Error sintáctico: se esperaba `;`
Variable no declarada	`print_int(numero);` sin declarar `numero`	Error semántico: variable no declarada
Tipos incompatibles	`int numero = "hola";`	Error semántico: tipos incompatibles
Carácter no reconocido	usar `.` para campos de struct	`K++: carácter no reconocido: '.'`
Declaración global inválida	código fuera de una función/ struct/enum	`se esperaba una declaración global (struct, enum o función int)`

> **Consejo:** lee el mensaje con calma: K++ te dice *qué* esperaba encontrar y normalmente el problema está en esa línea o en la anterior (casi siempre un `;` olvidado).

14. Compilar y ejecutar tu programa

Paso 1: escribe tu programa

Guarda tu código en un archivo con extensión `.kpp`, por ejemplo `mi_programa.kpp`.

Paso 2: compila

```
./bin/kpp mi_programa.kpp
```

El compilador ejecuta su pipeline completo: análisis léxico → sintáctico → semántico → generación de IR → backend x86 → emisor → escritor del ejecutable. En pantalla verás la cabecera `K++ COMPILER` y el AST de tu programa.

Paso 3: obtén el ejecutable

El resultado es un archivo `.bin`: un ejecutable nativo de **KolibriOS** (x86 de 32 bits, con su cabecera correspondiente). Cópialo a tu instalación o imagen de KolibriOS y ejecútalo allí.

> **Flujo de trabajo recomendado:** el repositorio incluye `kpp_bootstrap_studio.py`, un mini-IDE gráfico (editor + compilación + desensamblado del binario) para iterar rápido durante el desarrollo.

15. Limitaciones conocidas

K++ está en Stage 0. Esta tabla resume con honestidad lo que aún no hace, según la matriz de pruebas del proyecto:

Área	Estado
División y módulo (`/` `%`)	❌ No implementados: el compilador los rechaza con error en lugar de generar código incorrecto (L03)
Acceso a campos de `struct` (`p.edad`)	❌ Bloqueado: el lexer no reconoce `.` (L13)
Shadowing (variables locales que ocultan otras)	❌ Fallo registrado (L12)
Recursión	⏳ Pendiente de confirmar ejecución (L15)

Llamadas anidadas (`f(g(x))`)	❌ Falla en el backend x86 (L16)
Tipos `void`, `bool`, `float`, `double`, `char`	❌ No asumir soporte
Arreglos y punteros	❌ En hoja de ruta
Self-hosting (compilador escrito en K++)	⏳ Objetivo futuro

16. Referencia de ejercicios verificados L01–L16

Cada ejercicio de la suite oficial y su estado, tal como lo registra el proyecto. Los capítulos de este manual se construyeron sobre los ejercicios marcados ✅.

ID	Tema	Estado	Archivo de referencia
L01	Tipos `int` y `string`	✅ PASS	`test_l01_tipos.kpp`
L02	Variables e inicializadores	✅ PASS	`test_l02_variables.kpp`
L03	Operaciones aritméticas	⚠️ Parcial (`+ -` sí; `* / %` no)	`test_l03_aritmetica.kpp`
L04	Funciones y retornos	✅ PASS	`test_l04_funciones.kpp`
L05	Parámetros, argumentos y llamadas	✅ PASS	`test_l05_parametros.kpp`
L06	Impresión de literales	✅ PASS	`test_l06_impresion.kpp`
L07	`struct` (declaración)	✅ PASS	`test_l07_struct.kpp`
L08	`enum`	✅ PASS	`test_l08_enum.kpp`, `test_enum_valores.kpp`
L09	Operadores lógicos y relacionales	✅ PASS	`kpp-l09-operadores-audit-20261009.kpp`
L10	`if` / `else`	✅ PASS	`test_l10_if_else_audit_20261009.kpp`
L11	`while`	✅ PASS	`test_l11_while_audit_20261009.kpp`

L12	Ámbitos y shadowing	❌ FAIL (shadowing)	—
L13	Campos de `struct`	❌ Bloqueada (token `.`)	`test_l13_struct_campos.kpp`
L14	Errores sintácticos y semánticos	✅ PASS	`test_l14_errores.kpp`
L15	Recursión	⏳ Pendiente de confirmar	`test_l15_recursion.kpp`
L16	Llamadas anidadas	❌ FAIL en backend x86	—

Batería de pruebas de llamadas a funciones (verificada por auditoría de binario)

Nueve pruebas dedicadas a llamadas y parámetros, compiladas con `bin/kpp` y verificadas instrucción por instrucción en el desensamblado (`objdump -m i386 -Mintel` sobre el payload del `.bin`): cabecera `MENUET01` válida, `call`/`ret` balanceados, argumentos en EAX/EBX según la convención interna y aritmética correcta.

Archivo	Qué comprueba	Resultado esperado	Estado
`test_function_call_variables.kpp`	Pasar variables como argumentos	Retorno 30	✅ Verificada
`test_function_call_expression.kpp`	Expresión como argumento (`x + 5`)	Retorno 35	✅ Verificada
`test_function_call_two_expressions.kpp`	Expresiones en ambos argumentos (`x + 5`, y `* 2`)	Retorno 55	✅ Verificada
`test_function_call_forward.kpp`	Llamar función definida después de `main`	Imprimir 20	✅ Verificada
`test_function_call_store.kpp`	Guardar el retorno en una variable	Termina con retorno 0	✅ Verificada
`test_function_call_print.kpp`		Imprimir 10	

	Imprimir el retorno de una llamada		✅ Verificada
`test_function_call_noprint.kpp`	Retornar directamente el resultado de una llamada	Retorno 10	✅ Verificada
`test_function_parameters_use.kpp`	Usar parámetros en una suma	Retorno 30	✅ Verificada
`test_una_sola_llamada.kpp`	Llamada simple con dos argumentos	Imprimir 30	✅ Verificada

---

Manual generado a partir del código fuente real del proyecto K++ (repositorio `github.com/saykol31/KPP`), verificando cada ejemplo con el compilador `bin/kpp`, la matriz de pruebas L01–L16 y la auditoría de la batería de llamadas a funciones (octubre de 2026). K++ es software experimental: la sintaxis puede cambiar entre versiones.